Sincronização do Jantar dos Filósofos

Pedro Gripp

Sistemas de Tempo Real

29 de setembro de 2026

Problema e requisitos

- Cinco filósofos compartilham cinco garfos; cada filósofo precisa dos dois garfos adjacentes para comer.
- Estados dos filósofos: pensando (`P`), com fome (`F`), comendo (`C`).
- Estados dos garfos: livre (`L`) e ocupado (`O`).
- Requisito: sincronizar o acesso e observar as mudanças de estado no terminal.

Vetor de estados impresso a cada mudança

```
P,P,P,P,P,L,L,L,L,L
        ↓
F,P,P,P,F,O,L,L,L,O
```

A tupla contém cinco estados de filósofos seguidos por cinco estados de garfos.

Modelagem do sistema

- `Garfo`: protege o recurso com `std::mutex` e `try_lock()`.
- `Filosofo`: executa o ciclo em uma `std::thread`.
- `std::mt19937`: gera durações aleatórias de pensamento e refeição.
- `std::chrono`: mede o tempo acumulado à espera dos garfos.
- Um mutex global protege a atualização e impressão do vetor de estados.

Relacionamento entre objetos

```
Filosofo[1] - Garfo[1], Garfo[2]
Filosofo[2] - Garfo[2], Garfo[3]
⋮
Filosofo[5] - Garfo[5], Garfo[1]
```

Cada filósofo compartilha cada garfo com o vizinho correspondente.

Sincronização e prevenção de deadlock

```
if (esq  dir) return true;
if (esq) garfoEsq.soltar(); if (dir) garfoDir.soltar(); return false;
```

- A aquisição usa tentativas não bloqueantes, uma para cada garfo.
- Se não obtém os dois, libera qualquer garfo que tenha obtido antes de tentar novamente.
- Isso evita manter um garfo enquanto espera pelo outro, removendo a condição de retenção e espera associada ao deadlock.
- Em caso de falha, o código dorme por 10 ms antes de repetir, reduzindo busy waiting.

Metodologia e ferramentas

- Duração por execução: 5 minutos (300 000 ms)..
- Política RT: `SCHED_RR`, prioridade 80, se permitida no ambiente.

Comando para a execução RT

```
sudo chrt -rr 80 /usr/bin/time -v ./filosofo
```

- `/usr/bin/time -v`: trocas de contexto voluntárias e involuntárias do processo.

`SCHED_RR` não significa que o kernel seja `PREEMPT_RT`; registre a configuração do kernel separadamente.

Resultados da execução de cinco minutos

Filósofo	SCHED_RR Refeições	Espera (ms))
1	41	76.259,1
2	47	94.313,9
3	41	80.413,3
4	37	104.609,0
5	39	103.426,0
Soma	205	459.021,3

Trocas de contexto do processo

Voluntárias: 45900 Involuntárias: 0

Conclusões e Interpretação dos Resultados

- **Prevenção de Deadlock:** A abordagem não bloqueante (`try_lock`) com liberação imediata em caso de falha eliminou a espera circular, garantindo a execução contínua sem travamentos.
- **Equidade no Escalonamento:** Os filósofos apresentaram distribuição uniforme de refeições (37 a 47), confirmando a ausência de *starvation*.
- **Análise do Kernel (`0` preempções involuntárias):** O valor zero de preempções forçadas é coerente e correto, pois as threads realizam chamadas bloqueantes de temporização (`sleep_for`), cedendo voluntariamente o processador (`45.900` trocas voluntárias) muito antes do término da fatia de tempo (*quantum*) do `SCHED_RR`.

Trabalhos futuros

Substituir os `sleep_for` por carga computacional ativa (CPU-bound) para forçar o esgotamento do *quantum* do `SCHED_RR`, além de comparar o comportamento em kernel padrão versus `PREEMPT_RT`.